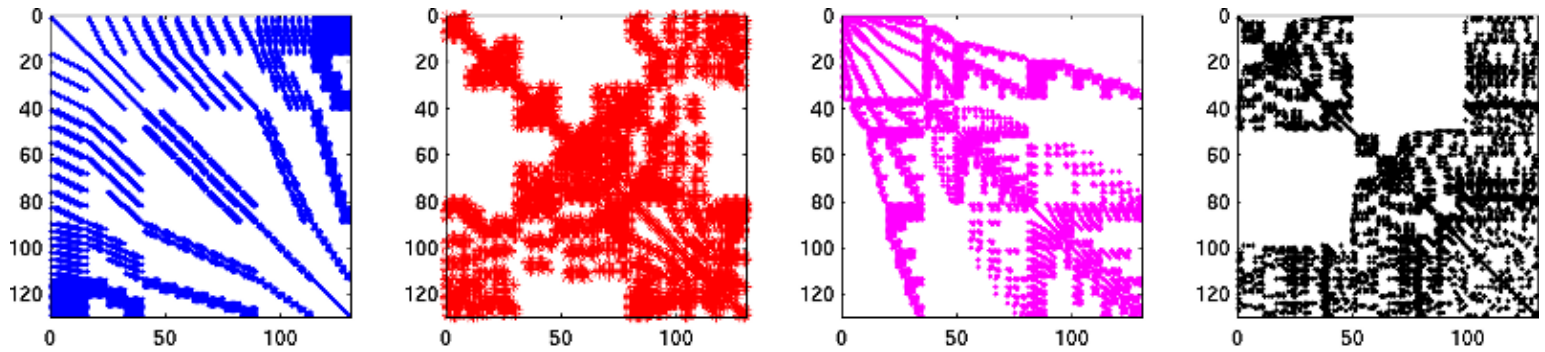


MA385 Part 3: Linear Algebra 1

3.1: Introduction to solving linear systems

Dr Niall Madden

22 October 2026

0.

1. Solving Linear Systems
2. A short review of linear algebra
3. The time it takes to compute det(A)
4. Exercises

For more, see Section 2.1 of Suli and Mayers:
https://ebookcentral.proquest.com/lib/nuig/reader.action?docID=221072&ppg=51&c=UERG

1. Solving Linear Systems

This section of the course is concerned with solving systems of linear equations ("simultaneous equations"). All problems are of the form: *find the set of real numbers* x_1, x_2, ..., x_n *such that*

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n \end{aligned}$$

where the a_{ij} *and* b_i *are real numbers.*
It is natural to rephrase this using the language of linear algebra:
Find $\mathbf{x} = (x_1, x_2, \ldots, x_n)^T \in \mathbb{R}^n$ *such that*

$$A\mathbf{x} = \mathbf{b}, \qquad \text{where } A \in \mathbb{R}^{n\times n}, \text{ and } \mathbf{b} \in \mathbb{R}^n. \tag{1}$$

1

1. Solving Linear Systems

In this section, we'll try to find a clever way to solving this system. In particular,

1. we'll argue that its unnecessary and (more importantly) expensive to try to compute A^{-1};
2. we'll have a look at **Gaussian Elimination**, but will study it in the context of LU-factorisation;
3. after a detour to talk about **matrix norms**, we calculate the **condition number** of a matrix;
4. this last task will require us to be able to estimate the **eigenvalues** (and singular values) of matrices: that is the main subject of the next and final section of the module.

2. A short review of linear algebra

- A vector $\mathbf{x} \in \mathbb{R}^n$, is an ordered n-tuple of real numbers, $\mathbf{x} = (x_1, x_2, \ldots, x_n)^T$.

- A matrix $A \in \mathbb{R}^{m \times n}$ is a rectangular array of n columns of m real numbers. (But we will only deal with **square** matrices, i.e., ones where $m = n$).

- Some square matrices are **symmetric**: we write $A = A^T$, meaning that $a_{i,j} = a_{j,i}$.

2. A short review of linear algebra

- You already know how to multiply a matrix by a vector, and a matrix by a matrix (right??).

- You can write the **scalar** product as $(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T\mathbf{y} = \sum_{i=1}^{n} x_i y_i$.

- Recall that $(A\mathbf{x})^T = \mathbf{x}^T A^T$, so
$(\mathbf{x}, A\mathbf{y}) = \mathbf{x}^T A\mathbf{y} = (A^T\mathbf{x})^T\mathbf{y} = (A^T\mathbf{x}, \mathbf{y})$.

2. A short review of linear algebra

- Letting I be the identity matrix, if there is a matrix B such that $AB = BA = I$, when we call B the **inverse** of A and write $B = A^{-1}$. If there is no such matrix, we say that A is **singular**.

- The scalar λ is called an **eigenvalue** of A if there is some vector, x for which $Ax = \lambda x$. We call x an **eigenvector**, corresponding to λ.
  A square $n \times n$ matrix can have up to n eigenvalues.

Theorem 3.1.1

The following statements are equivalent.

1. For **any** $\mathbf{b}$, $A\mathbf{x} = \mathbf{b}$ has a solution.
2. If there is a solution to $A\mathbf{x} = \mathbf{b}$, it is unique.
3. If $A\mathbf{x} = 0$ then $\mathbf{x} = 0$.
4. The columns (or rows) of A are linearly independent.
5. There exists a matrix $A^{-1} \in \mathbb{R}^{n \times n}$ such that $AA^{-1} = I = A^{-1}A$.
6. $\det(A) \neq 0$.
7. A has rank n.
8. If λ is an eigenvalue of A then $\lambda \neq 0$.

2. A short review of linear algebra

Item 5 of the above lists suggests that we could solve $A\mathbf{x} = \mathbf{b}$ as follows:

- find A^{-1}
- compute $\mathbf{x} = A^{-1}\mathbf{b}$.

However this is not the best way to proceed. We only need x and computing A^{-1} requires quite a bit a work.

3. The time it takes to compute det(A)

In introductory linear algebra courses, you might compute

$$A^{-1} = \frac{1}{\det(A)} A^*$$

where A^* is the adjoint matrix. But it turns out that computing $\det(A)$ directly can be **very** time-consuming.
Let d_n be the number of multiplications required to compute the determinant of an $n \times n$ matrix using the **Method of Minors** (also called "the Laplacian determinant expansion"). We know that if

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

then $\det(A) = ad - bc$. So $d_2 = 2$.

3. The time it takes to compute det(A)

If

$$A = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & j \end{pmatrix}$$

then

$$\det(A) = \begin{vmatrix} a & b & c \\ d & e & f \\ g & h & j \end{vmatrix} = a \begin{vmatrix} e & f \\ h & j \end{vmatrix} - b \begin{vmatrix} d & f \\ g & j \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix},$$

so $d_3 \geq 3d_2$.

3. The time it takes to compute det(A)

More generally, if A is an $n \times n$ matrix, we can compute its determinant as

$$\det(A) = \sum_{i=1}^{n} (-1)^{i-1} a_{1,i} \det(A^{(1,i)}).$$

where $A^{(1,i)}$ denotes the $(n-1) \times (n-1)$ submatrix of A obtained by deleting row 1, and column j.
Therefore, $d_n = n d_{n-1}$.

3. The time it takes to compute det(A)

In general we find that it takes $(n)(n-1)\cdots(4)(3)(2)(1) = n!$ operations to compute the determinant this way. And that is a lot of operations.

However, if you try this yourself using, say, Python/numpy (e.g., `np.linalg.det(np.random.randn(n, n))`) the you'd find the time taken scales more like n^3 than $n!$. Learning how that works is part of our next few classes.

4. Exercises

Exercise 3.1.1

Suppose you had a computer that can perform 2 billion operations per second.

1. Give an estimate for the number of seconds needed to compute the determinant of a 12-by-12 matrix, using the method of minors.
2. Estimate the largest n for which you could determine the determinant of an $n \times n$ matrix, using the method of minors, in a year.

Exercise 3.1.2

The Cauchy-Binet formula for determinants tells us that, if A and B are square matrices of the same size, then $\det(AB) = \det(A)\det(B)$. Using it, or otherwise, show that $\det(\sigma A) = \sigma^n \det(A)$ for any $A \in \mathbb{R}^{n\times n}$ and any scalar $\sigma \in \mathbb{R}$. *Note: this exercise gives us another reason to avoid trying to calculate the determinant of the coefficient matrix in order to find the inverse, and thus the solution to the problem. For example $A \in \mathbb{R}^{16\times 16}$ and the system is rescaled by $\sigma = 0.1$, then $\det(A)$ is rescaled by 10^{-16}. On standard computing platforms, like numpy/scipy/MATLAB, it would be indistinguishable from a singular matrix!*